

13.5 Dynamic Programming

In this section, we will discuss the *dynamic-programming* algorithmic design pattern. This technique is similar to the divide-and-conquer technique (Section 12.1.1), in that it can be applied to a wide variety of different problems. Dynamic programming can often be used to produce polynomial-time algorithms to solve problems that seem to require exponential time. In addition, the algorithms that result from applications of the dynamic programming technique are usually quite simple, often needing little more than a few lines of code to describe some nested loops for filling in a table.

13.5.1 Matrix Chain-Product

Rather than starting out with an explanation of the general components of the dynamic programming technique, we begin by giving a classic, concrete example. Suppose we are given a collection of n two-dimensional matrices for which we wish to compute the mathematical product

$$A = A_0 \cdot A_1 \cdot A_2 \cdots A_{n-1},$$

where A_i is a $d_i \times d_{i+1}$ matrix, for $i = 0, 1, 2, \dots, n-1$. In the standard matrix multiplication algorithm (which is the one we will use), to multiply a $d \times e$ -matrix B times an $e \times f$ -matrix C , we compute the product, A , as

$$A[i][j] = \sum_{k=0}^{e-1} B[i][k] \cdot C[k][j].$$

This definition implies that matrix multiplication is associative, that is, it implies that $B \cdot (C \cdot D) = (B \cdot C) \cdot D$. Thus, we can parenthesize the expression for A any way we wish and we will end up with the same answer. However, we will not necessarily perform the same number of primitive (that is, scalar) multiplications in each parenthesization, as is illustrated in the following example.

Example 13.8: Let B be a 2×10 -matrix, let C be a 10×50 -matrix, and let D be a 50×20 -matrix. Computing $B \cdot (C \cdot D)$ requires $2 \cdot 10 \cdot 20 + 10 \cdot 50 \cdot 20 = 10400$ multiplications, whereas computing $(B \cdot C) \cdot D$ requires $2 \cdot 10 \cdot 50 + 2 \cdot 50 \cdot 20 = 3000$ multiplications.

The *matrix chain-product* problem is to determine the parenthesization of the expression defining the product A that minimizes the total number of scalar multiplications performed. As the example above illustrates, the differences between parenthesizations can be dramatic, so finding a good solution can result in significant speedups.

Defining Subproblems

One way to solve the matrix chain-product problem is to simply enumerate all the possible ways of parenthesizing the expression for A and determine the number of multiplications performed by each one. Unfortunately, the set of all different parenthesizations of the expression for A is equal in number to the set of all different binary trees that have n leaves. This number is exponential in n . Thus, this straightforward (“brute-force”) algorithm runs in exponential time, for there are an exponential number of ways to parenthesize an associative arithmetic expression.

We can significantly improve the performance achieved by the brute-force algorithm, however, by making a few observations about the nature of the matrix chain-product problem. The first is that the problem can be split into **subproblems**. In this case, we can define a number of different subproblems, each of which is to compute the best parenthesization for some subexpression $A_i \cdot A_{i+1} \cdots A_j$. As a concise notation, we use $N_{i,j}$ to denote the minimum number of multiplications needed to compute this subexpression. Thus, the original matrix chain-product problem can be characterized as that of computing the value of $N_{0,n-1}$. This observation is important, but we need one more in order to apply the dynamic programming technique.

Characterizing Optimal Solutions

The other important observation we can make about the matrix chain-product problem is that it is possible to characterize an optimal solution to a particular subproblem in terms of optimal solutions to its subproblems. We call this property the **subproblem optimality** condition.

In the case of the matrix chain-product problem, we observe that, no matter how we parenthesize a subexpression, there has to be some final matrix multiplication that we perform. That is, a full parenthesization of a subexpression $A_i \cdot A_{i+1} \cdots A_j$ has to be of the form $(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$, for some $k \in \{i, i+1, \dots, j-1\}$. Moreover, for whichever k is the correct one, the products $(A_i \cdots A_k)$ and $(A_{k+1} \cdots A_j)$ must also be solved optimally. If this were not so, then there would be a global optimal that had one of these subproblems solved suboptimally. But this is impossible, since we could then reduce the total number of multiplications by replacing the current subproblem solution by an optimal solution for the subproblem. This observation implies a way of explicitly defining the optimization problem for $N_{i,j}$ in terms of other optimal subproblem solutions. Namely, we can compute $N_{i,j}$ by considering each place k where we could put the final multiplication and taking the minimum over all such choices.

Designing a Dynamic Programming Algorithm

We can therefore characterize the optimal subproblem solution, $N_{i,j}$, as

$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\},$$

where $N_{i,i} = 0$, since no work is needed for a single matrix. That is, $N_{i,j}$ is the minimum, taken over all possible places to perform the final multiplication, of the number of multiplications needed to compute each subexpression plus the number of multiplications needed to perform the final matrix multiplication.

Notice that there is a *sharing of subproblems* going on that prevents us from dividing the problem into completely independent subproblems (as we would need to do to apply the divide-and-conquer technique). We can, nevertheless, use the equation for $N_{i,j}$ to derive an efficient algorithm by computing $N_{i,j}$ values in a bottom-up fashion, and storing intermediate solutions in a table of $N_{i,j}$ values. We can begin simply enough by assigning $N_{i,i} = 0$ for $i = 0, 1, \dots, n-1$. We can then apply the general equation for $N_{i,j}$ to compute $N_{i,i+1}$ values, since they depend only on $N_{i,i}$ and $N_{i+1,i+1}$ values that are available. Given the $N_{i,i+1}$ values, we can then compute the $N_{i,i+2}$ values, and so on. Therefore, we can build $N_{i,j}$ values up from previously computed values until we can finally compute the value of $N_{0,n-1}$, which is the number that we are searching for. A Java implementation of this *dynamic programming* solution is given in Code Fragment 13.6; we use techniques from Section 3.1.5 for working with a two-dimensional array in Java.

```

1 public static int[ ][ ] matrixChain(int[ ] d) {
2     int n = d.length - 1;                // number of matrices
3     int[ ][ ] N = new int[n][n];         // n-by-n matrix; initially zeros
4     for (int b=1; b < n; b++)             // number of products in subchain
5         for (int i=0; i < n - b; i++) {   // start of subchain
6             int j = i + b;               // end of subchain
7             N[i][j] = Integer.MAX_VALUE; // used as 'infinity'
8             for (int k=i; k < j; k++)
9                 N[i][j] = Math.min(N[i][j], N[i][k] + N[k+1][j] + d[i]*d[k+1]*d[j+1]);
10        }
11    return N;
12 }
```

Code Fragment 13.6: Dynamic programming algorithm for the matrix chain-product problem.

Thus, we can compute $N_{0,n-1}$ with an algorithm that consists primarily of three nested loops (the third of which computes the min term). Each of these loops iterates at most n times per execution, with a constant amount of additional work within. Therefore, the total running time of this algorithm is $O(n^3)$.

13.5.2 DNA and Text Sequence Alignment

A common text-processing problem, which arises in genetics and software engineering, is to test the similarity between two text strings. In a genetics application, the two strings could correspond to two strands of DNA, for which we want to compute similarities. Likewise, in a software engineering application, the two strings could come from two versions of source code for the same program, for which we want to determine changes made from one version to the next. Indeed, determining the similarity between two strings is so common that the Unix and Linux operating systems have a built-in program, named `diff`, for comparing text files.

Given a string $X = x_0x_1x_2 \cdots x_{n-1}$, a **subsequence** of X is any string that is of the form $x_{i_1}x_{i_2} \cdots x_{i_k}$, where $i_j < i_{j+1}$; that is, it is a sequence of characters that are not necessarily contiguous but are nevertheless taken in order from X . For example, the string `AAAG` is a subsequence of the string `CGATAAATTGAGA`.

The DNA and text similarity problem we address here is the **longest common subsequence** (LCS) problem. In this problem, we are given two character strings, $X = x_0x_1x_2 \cdots x_{n-1}$ and $Y = y_0y_1y_2 \cdots y_{m-1}$, over some alphabet (such as the alphabet $\{A, C, G, T\}$ common in computational genomics) and are asked to find a longest string S that is a subsequence of both X and Y . One way to solve the longest common subsequence problem is to enumerate all subsequences of X and take the largest one that is also a subsequence of Y . Since each character of X is either in or not in a subsequence, there are potentially 2^n different subsequences of X , each of which requires $O(m)$ time to determine whether it is a subsequence of Y . Thus, this brute-force approach yields an exponential-time algorithm that runs in $O(2^n m)$ time, which is very inefficient. Fortunately, the LCS problem is efficiently solvable using **dynamic programming**.

The Components of a Dynamic Programming Solution

As mentioned above, the dynamic programming technique is used primarily for **optimization** problems, where we wish to find the “best” way of doing something. We can apply the dynamic programming technique in such situations if the problem has certain properties:

Simple Subproblems: There has to be some way of repeatedly breaking the global optimization problem into subproblems. Moreover, there should be a way to parameterize subproblems with just a few indices, like i , j , k , and so on.

Subproblem Optimization: An optimal solution to the global problem must be a composition of optimal subproblem solutions.

Subproblem Overlap: Optimal solutions to unrelated subproblems can contain subproblems in common.

Applying Dynamic Programming to the LCS Problem

Recall that in the LCS problem, we are given two character strings, X and Y , of length n and m , respectively, and are asked to find a longest string S that is a subsequence of both X and Y . Since X and Y are character strings, we have a natural set of indices with which to define subproblems—indices into the strings X and Y . Let us define a subproblem, therefore, as that of computing the value $L_{j,k}$, which we will use to denote the length of a longest string that is a subsequence of both the first j characters of X and the first k characters of Y , that is of prefixes $X[0..j-1]$ and $Y[0..k-1]$. If either $j = 0$ or $k = 0$, then $L_{j,k}$ is trivially defined as 0.

When both $j \geq 1$ and $k \geq 1$, this definition allows us to rewrite $L_{j,k}$ recursively in terms of optimal subproblem solutions. This definition depends on which of two cases we are in. (See Figure 13.13.)

- $x_{j-1} = y_{k-1}$. In this case, we have a match between the last character of $X[0..j-1]$ and the last character of $Y[0..k-1]$. We claim that this character belongs to a longest common subsequence of $X[0..j-1]$ and $Y[0..k-1]$. To justify this claim, let us suppose it is not true. There has to be some longest common subsequence $x_{a_1}x_{a_2}\dots x_{a_c} = y_{b_1}y_{b_2}\dots y_{b_c}$. If $x_{a_c} = x_{j-1}$ or $y_{b_c} = y_{k-1}$, then we get the same sequence by setting $a_c = j-1$ and $b_c = k-1$. Alternately, if $x_{a_c} \neq x_{j-1}$ and $y_{b_c} \neq y_{k-1}$, then we can get an even longer common subsequence by adding $x_{j-1} = y_{k-1}$ to the end. Thus, a longest common subsequence of $X[0..j-1]$ and $Y[0..k-1]$ ends with x_{j-1} . Therefore, we set

$$L_{j,k} = 1 + L_{j-1,k-1} \quad \text{if } x_{j-1} = y_{k-1}.$$

- $x_{j-1} \neq y_{k-1}$. In this case, we cannot have a common subsequence that includes both x_{j-1} and y_{k-1} . That is, we can have a common subsequence end with x_{j-1} or one that ends with y_{k-1} (or possibly neither), but certainly not both. Therefore, we set

$$L_{j,k} = \max\{L_{j-1,k}, L_{j,k-1}\} \quad \text{if } x_{j-1} \neq y_{k-1}.$$

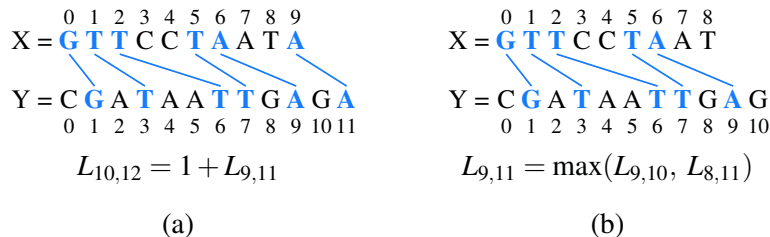


Figure 13.13: The two cases in the longest common subsequence algorithm for computing $L_{j,k}$ when $j, k \geq 1$: (a) $x_{j-1} = y_{k-1}$; (b) $x_{j-1} \neq y_{k-1}$.

The LCS Algorithm

The definition of $L_{j,k}$ satisfies subproblem optimization, for we cannot have a longest common subsequence without also having longest common subsequences for the subproblems. Also, it uses subproblem overlap, because a subproblem solution $L_{j,k}$ can be used in several other problems (namely, the problems $L_{j+1,k}$, $L_{j,k+1}$, and $L_{j+1,k+1}$). Turning this definition of $L_{j,k}$ into an algorithm is actually quite straightforward. We create an $(n+1) \times (m+1)$ array, L , defined for $0 \leq j \leq n$ and $0 \leq k \leq m$. We initialize all entries to 0, in particular so that all entries of the form $L_{j,0}$ and $L_{0,k}$ are zero. Then, we iteratively build up values in L until we have $L_{n,m}$, the length of a longest common subsequence of X and Y . We give a Java implementation of this algorithm in Code Fragment 13.7.

```

1  /** Returns table such that L[j][k] is length of LCS for X[0..j-1] and Y[0..k-1]. */
2  public static int[ ][ ] LCS(char[ ] X, char[ ] Y) {
3      int n = X.length;
4      int m = Y.length;
5      int[ ][ ] L = new int[n+1][m+1];
6      for (int j=0; j < n; j++)
7          for (int k=0; k < m; k++)
8              if (X[j] == Y[k])           // align this match
9                  L[j+1][k+1] = L[j][k] + 1;
10             else                         // choose to ignore one character
11                 L[j+1][k+1] = Math.max(L[j][k+1], L[j+1][k]);
12     return L;
13 }
```

Code Fragment 13.7: Dynamic programming algorithm for the LCS problem.

The running time of the algorithm of the LCS algorithm is easy to analyze, for it is dominated by two nested **for** loops, with the outer one iterating n times and the inner one iterating m times. Since the if-statement and assignment inside the loop each requires $O(1)$ primitive operations, this algorithm runs in $O(nm)$ time. Thus, the dynamic programming technique can be applied to the longest common subsequence problem to improve significantly over the exponential-time brute-force solution to the LCS problem.

The LCS method of Code Fragment 13.7 computes the length of the longest common subsequence (stored as $L_{n,m}$), but not the subsequence itself. Fortunately, it is easy to extract the actual longest common subsequence if given the complete table of $L_{j,k}$ values computed by the LCS method. The solution can be reconstructed back to front by reverse engineering the calculation of length $L_{n,m}$. At any position $L_{j,k}$, if $x_j = y_k$, then the length is based on the common subsequence associated with length $L_{j-1,k-1}$, followed by common character x_j . We can record x_j as part of the sequence, and then continue the analysis from $L_{j-1,k-1}$. If $x_j \neq y_k$, then we can move to the larger of $L_{j,k-1}$ and $L_{j-1,k}$. We continue this process until reaching

some $L_{j,k} = 0$ (for example, if j or k is 0 as a boundary case). A Java implementation of this strategy is given in Code Fragment 13.8. This method constructs a longest common subsequence in $O(n + m)$ additional time, since each pass of the **while** loop decrements either j or k (or both). An illustration of the algorithm for computing the longest common subsequence is given in Figure 13.14.

```

1  /** Returns the longest common substring of X and Y, given LCS table L. */
2  public static char[] reconstructLCS(char[] X, char[] Y, int[][] L) {
3      StringBuilder solution = new StringBuilder();
4      int j = X.length;
5      int k = Y.length;
6      while (L[j][k] > 0)                                // common characters remain
7          if (X[j-1] == Y[k-1]) {
8              solution.append(X[j-1]);
9              j--;
10             k--;
11         } else if (L[j-1][k] >= L[j][k-1])
12             j--;
13         else
14             k--;
15     // return left-to-right version, as char array
16     return solution.reverse().toString().toCharArray();
17 }

```

Code Fragment 13.8: Reconstructing the longest common subsequence.

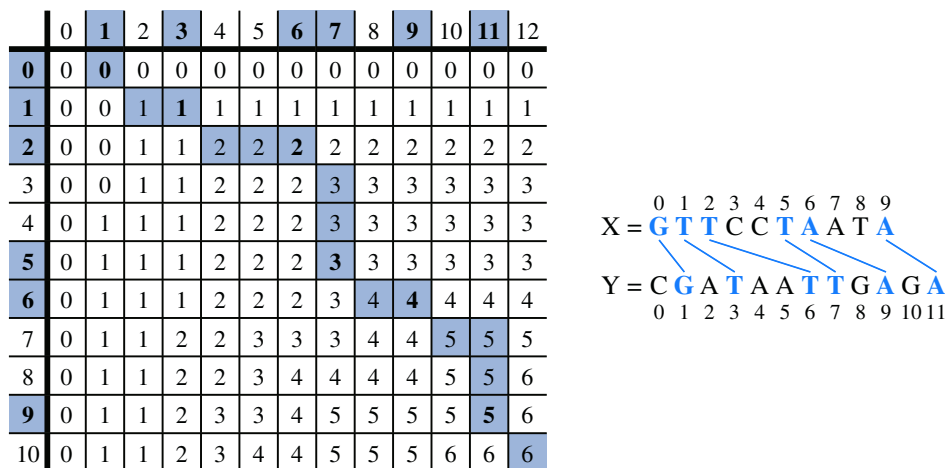


Figure 13.14: Illustration of the algorithm for constructing a longest common subsequence from the array L . A diagonal step on the highlighted path represents the use of a common character (with that character's respective indices in the sequences highlighted in the margins).